

コスパチェッカー 設計ドキュメント

要件定義 / データベース設計 / ルーティング設計 — MVP (Phase 1) ベース・拡張対応

● 要件定義

MUST

DPコスバ計算エンジン

胃袋・滞在時間を制約としたナップサックDP。好み重みづけ込みのスコアを最大化する皿の組み合わせを算出する。

MUST

食べ放題 vs 単品 判定

最適解の単品合計と食べ放題料金を比較し結果を表示。差額（節約額 or 損失額）も明示する。

MUST

メニュー管理

プリセットメニューから選択 + カスタムメニューを追加可能。名前・価格・重量・提供時間・好み度を設定。

MUST

結果表示画面

最適な皿の組み合わせ一覧 + 判定結果 + 差額を表示。視覚的にわかりやすいレイアウト。

NICE

ユーザー認証

Firebase Authでログイン。計算履歴をユーザーに紐づけて保存・閲覧。Phase 2で追加。

NICE

履歴保存・閲覧

Firestoreに計算結果を保存。履歴一覧画面から過去の結果を見返せる。Phase 2で追加。

NICE

Gemini解説文

結果画面にGemini生成の自然言語解説を追加。Phase 3で追加。

対象外

ソーシャル機能

他ユーザーとの共有・ランキング等は今回スコープ外。

開発フェーズ方針：Phase 1（MUST）を確実に完成させてデモできる状態にする。認証・履歴・GeminiはPhase 2/3で追加。設計は最初から拡張を想定した構造にする。

● データベース設計（FIRESTORE）

menus / menus / {menuId}

Phase 1

フィールド名	型	説明	必須
menuId	string	ドキュメントID（自動生成）	✓
name	string	メニュー名（例：カルビ、ハラミ）	✓

フィールド名	型	説明	必須
price	number	単品価格（円）	✓
weight	number	1皿あたりの重量（g）	✓
servingTime	number	食べるのにかかる時間（分）	✓
isPreset	boolean	プリセットか否か（true=プリセット）	✓
category	string	カテゴリ（肉・海鮮・サイドなど）拡張用	
createdBy	string null	作成ユーザーID。プリセットはnull（Phase 2拡張）	
createdAt	timestamp	作成日時	✓

sessions / sessions / {sessionId}

Phase 1

1回の計算セッションを1ドキュメントで管理。Phase 1ではゲストセッションとして使用。Phase 2でuserId紐づけに拡張。

フィールド名	型	説明	必須
sessionId	string	ドキュメントID（自動生成）	✓
userId	string null	Phase 1はnull、Phase 2でユーザー紐づけ	
input.stomachCapacity	number	胃袋容量（g）	✓
input.stayTime	number	滞在時間（分）	✓
input.buffetPrice	number	食べ放題の料金（円）	✓
input.selectedMenus[].menuId	string	選択したメニューのID	✓
input.selectedMenus[].preference	number	好み度スコア（1～5）	✓
result.optimalCombination	array	最適な皿の組み合わせ [{menuId, quantity}]	✓
result.totalPrice	number	単品合計金額（円）	✓
result.verdict	string	"buffet" or "a la carte"	✓
result.difference	number	差額（円）。正=食べ放題お得、負=単品お得	✓
result.geminiComment	string null	Gemini生成コメント（Phase 3で追加）	
createdAt	timestamp	計算日時	✓

users / users / {userId}

Phase 2

フィールド名	型	説明
userId	string	Firebase AuthのUID
displayName	string	表示名
email	string	メールアドレス
createdAt	timestamp	登録日時

拡張ポイント：Phase 1からsessionsのuserIdフィールドをnullで持つておくことで、Phase 2の認証追加時にuserId紐づけだけ追加すれば履歴機能が実現できる。コレクション構造の変更不要。

- ルーティング設計 (REACT ROUTER)

/

トップ画面 — 胃袋・滞在時間・食べ放題料金の入力フォーム

Phase 1

/menu

メニュー選択画面 — プリセット一覧から選択 + カスタムメニュー追加 + 好み度設定

Phase 1

/result/:sessionId

結果画面 — 判定結果・最適組み合わせ・差額表示。
sessionIdでFirestoreから結果を取得

Phase 1

/login

ログイン画面 — Firebase Auth (Google認証)

Phase 2

/history

履歴一覧画面 — 過去の計算結果一覧。ログイン必須。

Phase 2

/history/:sessionId

履歴詳細。/result/:sessionIdと同じコンポーネントを使い
回し可能

Phase 2

/api-compare

APIサブスク比較モード — 従量課金 vs サブスクをDPで比較

Phase 3

画面遷移 (**Phase 1**) : / → /menu → /result/:sessionId

拡張ポイント : /result/:sessionIdのsessionIdをURLに持つことで、Phase 2の履歴機能で同じコンポーネントをそのまま使い回せる。認証ガードはPhase 2で/historyにだけ追加すればOK。

- コンポーネント構成 (参考)

```
// src/ └── components/ | └── ui/ // 汎用UIパーツ (Button, Input, Card等) | └──
MenuCard.tsx // メニュー1件のカード | └── ResultCard.tsx // 結果表示カード | └── Layout.tsx //
共通レイアウト └── pages/ | └── TopPage.tsx // / | └── MenuPage.tsx // /menu | └──
ResultPage.tsx // /result/:sessionId └── hooks/ | └── useDP.ts // DPロジック (純粋関数) |
└── useFirestore.ts // Firestore CRUD └── lib/ | └── dp.ts // ナップサックDPコア実装 |
└── firebase.ts // Firebase初期化 └── types/ └── index.ts // 型定義 (Menu, Session, Result
等)
```